

Programming Languages

Stuff to know

1. Question Types:
 - a. True/False (given a statement, evaluate it to be true or false)
 - b. Matching terms with descriptions
 - c. Short essay problems.
 - d. Code analysis problems.
2. Chapters covered include Chapters 1, 3, 4, 11, 12, and 15.
3. The exam is closed-book and closed-notes. However, you will be allowed to use up to 2 sheets (size 8 1/2x11, one page, front and back; or two pages, on one side only, typed or hand-written) of notes.

Text-book readings:

Chapters 1 & 2

Why Study Programming Languages

Language Evaluation Criteria

Readability (factors affect readability)

- Feature multiplicity
- Operator overloading
- Orthogonality
- Control structures
- Data Types and Structures

Writability (factors affect writability)

- Expressivity
- Support for abstraction
- Simplicity and orthogonality

Reliability (factors affect reliability)

- Type checking
- Exception handling
- Aliasing
- Readability and writability

Language Categories

Main features for each language category

Representative languages in each category/application domain

Implementation Methods

Understand different implementation methods

A Brief History (minimum)

Other issues

- Language development with respect to applications
- Language standardization
- Internationalization

Chapter 3 Describing Syntax and Semantic

Introduction

- syntax and semantics

Syntax

- Basic syntactic concepts in a programming language
- Formal Method for Describing Syntax
 - o Grammar, Context-Free Grammar, or BNF
 - o Sentence, language, lexeme, token
 - o BNF notation for grammar

- o Derivations
- o Parsing
- o Ambiguity in Grammar
- o Operator Precedence
- o Associativity of Operators
- o Extended BNF

Semantic

- **Describing Static Semantics**
 - o Attribute Grammars
- **Describing (Dynamic) Semantics**
 - o Operational Semantics
 - o Axiomatic Semantics
 - o Denotational Semantics

Chapter 4 Lexical and Syntax Analysis

A Brief History

The big picture of a compiler and what the main functionalities of major components are.

The lexical analyzer

- Introduction
- What does a lexical analyzer do?
- How to build a lexical analyzer?
 - o State transition diagram

The Syntax Analyzer (The Parser)

- Introduction
- What do parsers do?
 - o Top-down
 - o Bottom-up
 - o Recursive Descent Parsers

The Semantic Analyzer

- the main functionalities

Intermediate Code Generator

- the main functionalities

Code Optimizer

- the main functionalities

Code Generator

- the main functionalities

Chapter 11 Abstract Data Types and Encapsulation Concepts

The Concept of Abstraction

- process abstraction and data abstraction

Define abstract data types

- two conditions
- interface and implementation
- Advantages of Data Abstraction
- Information hiding and encapsulation

- Design issues

ADT in C++

- the form of encapsulation
- how access controls are provided, i.e. access modifiers
- How C++ can separate interface from implementation
- Can ADT be parameterized? i.e. C++ templates
- Other Encapsulation constructs supported in C++

ADT in Java

- the form of encapsulation
- how access controls are provided, i.e. access modifiers
- How Java can separate interface from implementation
- Other encapsulation constructs supported by Java
- Can ADT be parameterized? i.e. Java generics

Chapter 12 Inheritance and Polymorphism

Introduction

- two issues addressed by inheritance
- disadvantage(s) of inheritance
- some related concepts
- single inheritance and multiple inheritance

Method overloading and method overriding

Abstract classes

Dynamic binding

- what is dynamic binding; polymorphism
- what is the main reason(s) for languages to support dynamic binding?

Inheritance in Java

- General characteristics
- constructor calls
- protected keywords
- the final keywords
- Multiple inheritance

Inheritance in C++

- general characteristics
- public and private inheritance; reexportation
- constructor calls
- "virtual" and dynamic binding

Inheritance in Python

- general characteristics
- properties
- inheritance
- abstract classes

Implementing OO constructs

- Class instance records
- Dynamic binding of method calls

Chapter 15 Functional Programming Languages

Language Paradigms

Unique features of functional programming languages

PROGRAMMING WITH SCHEME:

- Data types

- Atoms and Lists

- Basic Arithmetic

- Predicate Functions

- Processing Lists

- The LAMBDA function

- Defining Stuff

- Flow of Control

- High order functions

- Tail-recursion